Final Project – Cloud Computing and Virtualization

MEI-IoT – Masters in Informatics Engineering: Internet of Things

Prof. Renato Panda (renato.panda@ipt.pt)

2026-04-16

Table of contents

1 Final Project – Cloud Computing and Virtualization

1.1 Introduction

The project is designed to consolidate your understanding of modern cloud and virtualization technologies, focusing on the design, deployment, and critical analysis of scalable and resilient infrastructures and applications.

1.2 Learning Outcomes

Upon successful completion, you will be able to:

- Analyze and critique application architectures for scalability and resilience.
- Design and implement distributed systems using VMs and containers.
- Apply cloud-native and automation tools to real-world scenarios.
- Justify architectural decisions and reflect on trade-offs.

1.3 Objectives

By completing this project, you will:

- Develop practical skills in:
  - Provisioning tools
  - Load balancing
  - High availability
  - Service discovery
  - Containers and cloud-native technologies
  - Scalability and elasticity
- Learn to critically analyze and improve real-world application architectures.
- Demonstrate the ability to justify design decisions and reflect on trade-offs.

1.4 Project Context

Modern infrastructures must be resilient and capable of horizontal scaling to fully leverage virtualization and cloud computing. In this project, you will start with **Project O** — a simple PHP web application provided as a baseline Vagrant setup — and reimagine its deployment so that it becomes **scalable, resilient, and highly available**.

You will explore two distinct approaches (A and B), each emphasizing different aspects of cloud, containerization, and virtualization.

Project O (the original application) includes:

- Static content
- Data loading from external APIs (e.g., cat pictures)
- JavaScript libraries
- Sessions
- File uploads
- Relational database
- WebSocket communication

1.5 Group Work

- **Default:** The project should be done individually, to ensure hands-on experience with all tools and concepts.
- **Exception:** Groups of 2 students are allowed if justified (please contact the instructor in advance).

1.6 Project Tasks

You will design, implement, and analyze two alternative architectures for the provided web application:

1.6.1 A. Virtual Machines (On-Premises IaaS Simulation)

- Use **Vagrant** and virtual machines to deploy the application in a more robust, distributed, and scalable manner, simulating an on-premises IaaS environment.
- Use **Ubuntu** or **Alpine Linux** (recommended to save resources) as the VM operating system.
- You are free to explore and combine any of the tools and techniques covered in class, including (but not limited to):
  - **Automation:** Ansible (provisioning, configuration management)
  - **Load balancing:** nginx (or similar)
  - **High availability:** Keepalived (VRRP failover for load balancers)
  - **Database HA:** repmgr (PostgreSQL streaming replication), Patroni (automatic PostgreSQL failover)
  - **Sessions:** centralized session storage (e.g., Redis, Memcached)
  - **Storage:** shared/distributed storage for file uploads (e.g., GlusterFS, NFS)
  - **Service discovery:** Consul (automatic registration and load balancer updates)
  - **Monitoring & Observability:** Prometheus, Grafana, Loki, SigNoz

1.6.2 B. Containers & Cloud-Native (On-Premises or Public Cloud)

- Use **Docker containers** and orchestration (Docker Swarm, Kubernetes, or similar) to deploy the application, either on-premises or using a public cloud provider.
- Students will have access to **Google Cloud Platform (GCP) education credits** to explore managed cloud services (Compute Engine, Cloud SQL, Cloud Storage, GKE, Cloud Load Balancing, etc.).
- You may also explore **GCP emulators** locally before deploying to the cloud:
  - gcloud emulators

  - gcloud beta emulators
  - fake-gcs-server (GCS emulator)
  - fullstorydev/emulators (Pub/Sub, Bigtable, etc.)
- Integrate IaaS or cloud services as appropriate for your design. The same topics listed for Option A are valid here, applied in a container/cloud context.

For both approaches, you are expected to:

- Critically analyze **Project O** and its original architecture.
- Propose and implement an improved, scalable, resilient, and highly available architecture.
- Justify your design choices and discuss any limitations or trade-offs.

1.7 Deliverables

Submit a **ZIP file** containing all relevant materials:

- **Project Report** (recommended structure):
  1. **Introduction and Objectives**
  2. **Analysis of the Original Application**
     - System architecture diagram
     - Technologies used
     - Identified issues and limitations
     - (Optional) Performance metrics (e.g., using `hey` or `Vegeta`)
  3. **Proposed Solutions**
     - Separate sections for A (VMs) and B (Containers/Cloud)
     - Planned/ideal vs. implemented architecture (with architecture diagrams)
     - Main components, rationale, and implementation details
     - (Optional) Performance metrics
  4. **Discussion and Conclusions**
     - Key takeaways and lessons learned
     - Brief comparison: Original vs. A vs. B
- **Configuration/code and instructions** to replicate/run your solution.
- **Slides and demo** (max 15 minutes; slides may be submitted after the report).

1.8 Submission Guidelines

- Submit your ZIP file via the platform or as instructed by the professor.
- Ensure all files are clearly named and organized.
- Include a README with student name(s), instructions, and any special notes.

1.9 Suggestions and Tips

- **Critical Thinking:**
  Discuss features you considered, even if not implemented. Explain your reasoning and trade-offs.
- **Potential Features to Explore:**
  - Three-layer architecture (load balancer, web, database)
  - Serving the website correctly (partially or fully)
  - Manual or semi-automatic scaling of instances
  - Automatic service discovery and dynamic load balancer updates (e.g., Consul)
  - Auto-scaling the web layer (e.g., based on schedule or CPU load)
  - High availability for the load balancer (e.g., Keepalived / VRRP, cloud LB)
  - High availability for the database (e.g., repmgr, Patroni, managed cloud DB)
  - Centralized session storage to support horizontal web scaling (e.g., Redis)
  - Shared/distributed storage for file uploads (e.g., GlusterFS, NFS, cloud object storage)
  - Monitoring & observability stack (e.g., Prometheus + Grafana, Loki, SigNoz)
  - Automated deployment and configuration management (e.g., Ansible, Terraform/OpenTofu)
- **Documentation:**
  Use diagrams, tables, and clear explanations. Reference all external resources and tools used.

1.10 Resources

- **Project O repository:** https://github.com/renatopanda/ipt_cloud_course_project *(link may be updated — check the course platform for the latest version)*
- **Recommended tools:** Vagrant, Docker, Google Cloud Platform, and tools explored during classes: Ansible, nginx, Consul, Keepalived, repmgr, Patroni, Redis, GlusterFS, Prometheus, Grafana, Loki, SigNoz, Docker Swarm, Kubernetes, Terraform/OpenTofu, hey, Vegeta, etc.
- **GCP Emulators (Option B — local development):**
  - gcloud emulators
  - gcloud beta emulators
  - fake-gcs-server (GCS emulator)
  - fullstorydev/emulators (Pub/Sub, Bigtable, etc.)

1.11 Assessment Criteria

Your project will be evaluated based on the following **objective criteria**:

- **Analysis and Planning (15%)**
  - Depth and clarity of the original system analysis
  - Identification of relevant issues and requirements
- **Design and Implementation (35%)**
  - Appropriateness and justification of architectural choices
  - Correctness and completeness of the implemented solutions (A & B)
  - Use of relevant tools and technologies
- **Documentation and Communication (20%)**
  - Quality, clarity, and organization of the report and diagrams
  - Clear, reproducible instructions and code/configuration
- **Critical Reflection (15%)**
  - Discussion of limitations, trade-offs, and lessons learned
  - Comparison between original and proposed solutions
- **Presentation and Defense (15%)**
  - Clarity and effectiveness of the oral presentation/demo
  - Ability to answer questions and demonstrate understanding

All students must attend the project defense and demonstrate a clear understanding of the solution.

1.12 Deadline

To be defined — check the course platform or the next class session for the exact submission date.

For reference and updates, please consult the project repository and the course platform.